

ICCD633 CONSTRUCCIÓN Y EVOLUCIÓN DE SOFTWARE

Versión 1.2.0

INTEGRANTES

Julio Arrobo
Angel Anguaya
Juan Cofre
Jhordy Parra

FECHA:

27/11/2025

Contenido

1	Introducción	2
1.1	Propósito del documento	2
2	Herramientas de desarrollo	2
3	Librerías y dependencias.....	2
4	Configuración del proyecto	3
4.1	Estructura de carpetas	3
4.2	Configuración de JavaFX.....	3
4.3	Configuración de la herramienta de construcción	3
5	Flujo de ramas de GitHub Flow	5
6	Procedimiento de instalación	6
7	Javadoc	6
7.1	Etiquetas claves a documentar:.....	7

ICCD633 CONSTRUCCIÓN Y EVOLUCIÓN DE SOFTWARE

Versión 1.2.0

1 Introducción

1.1 Propósito del documento

El propósito de este documento es describir de manera detallada el entorno de desarrollo, las herramientas, dependencias, configuraciones y procedimientos necesarios para la correcta instalación, ejecución y mantenimiento de la aplicación de traducción de texto a Braille y viceversa, adicionalmente la descarga del documento en espejo. Además, proporciona lineamientos para la organización del proyecto bajo el patrón arquitectónico MVC, el uso de JavaFX, el flujo de trabajo con GitHub y la generación de documentación mediante Javadoc. Su objetivo es servir como una guía técnica clara y completa para desarrolladores actuales y futuros que participen en la construcción o evolución del sistema.

2 Herramientas de desarrollo

- Lenguaje de programación: Java (OpenJDK 25.0.1)
- Entorno de desarrollo integrado (IDE): Visual Studio Code
- Sistema de control de versiones: GitHub
- Herramienta de construcción y gestión de dependencias: Maven

3 Librerías y dependencias

Javafx-controls: Biblioteca que contiene la mayor parte de los componentes de la interfaz de usuario (UI). Incluye todos los widgets o controles estándar que un usuario puede ver e interactuar, como botones, etiquetas (labels), campos de texto (TextAreas), y layouts (contenedores).

Javafx-fxml: Biblioteca que implementa el lenguaje FXML, el cual es un lenguaje basado en XML que se utiliza para definir la estructura de la interfaz de usuario separada de la lógica de Java.

Maven-compiler-plugin: Plugin estándar de Maven para compilar el código fuente de Java. Su función principal es tomar los archivos .java del proyecto (Modelo, Vista, Controlador) y traducirlos a archivos de bytecode

Javafx-maven-plugin: Plugin especializado para proyectos JavaFX. Su rol principal es facilitar la ejecución del proyecto en el entorno de desarrollo y, crucialmente, el empaquetado de la aplicación.

Itexpdf:kernel: Contiene toda la infraestructura base para crear y manipular documentos PDF, manejar objetos del estándar PDF y escribir y leer archivos PDF a bajo nivel.

ICCD633 CONSTRUCCIÓN Y EVOLUCIÓN DE SOFTWARE

Versión 1.2.0

Itexpdf:layout: Permite manejar contenido visual de alto nivel dentro del PDF, como los párrfo, tablas, imágenes, etc.

Itexpdf:io: Gestiona las partes relacionadas con las entradas y salidas de archivos, manejo de fuentes personalizadas, lectura y escritura de imágenes.

4 Configuración del proyecto

4.1 Estructura de carpetas

Estructura principal del código siguiendo el patrón MVC.

- src\main\java\com\traductor\controller: contiene la lógica de manejo de eventos: TraductorController.
- src\main\java\com\traductor\model: contiene la lógica del negocio: TraductorBraille, DiccionarioBraille, SimboloBraille, ITraductor.
- src\main\java\com\traductor\view: contiene la clase principal de la aplicación: App.
- src\main\java\com\traductor\sevice: Contiene los servicios del sistema los cuales están relacionados con procesos específicos: GeneradorPDF, LectorPDF.
- src\main\resources\com\traductor: contiene al arquitecto de interiores para definir cómo se vera la interfaz: traductor.fxml .

4.2 Configuración de JavaFX

El proyecto consiste en el desarrollo de una aplicación capaz de convertir textos en español a su representación en Braille y viceversa, también debe permitir descargar la traducción en forma normal y en espejo. Para su implementación se utilizará el patrón arquitectónico Modelo-Vista-Controlador (MVC), el cual permitirá separar la lógica de negocio de la interfaz gráfica, facilitando la mantenibilidad, escalabilidad y evolución del sistema. La interfaz de usuario será desarrollada utilizando JavaFX, aprovechando sus capacidades para construir aplicaciones de escritorio modernas y estructuradas.

4.3 Configuración de la herramienta de construcción

ICCD633 CONSTRUCCIÓN Y EVOLUCIÓN DE SOFTWARE

Versión 1.2.0

```
15 <properties>
16   <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
17   <maven.compiler.source>17</maven.compiler.source>
18   <maven.compiler.target>17</maven.compiler.target>
19   <javafx.version>21.0.1</javafx.version>
20 </properties>
21
22 <dependencies>
23   <dependency>
24     <groupId>org.openjfx</groupId>
25     <artifactId>javafx-controls</artifactId>
26     <version>${javafx.version}</version>
27   </dependency>
28   <dependency>
29     <groupId>org.openjfx</groupId>
30     <artifactId>javafx-fxml</artifactId>
31     <version>${javafx.version}</version>
32   </dependency>
33   <dependency>
34     <groupId>com.itextpdf</groupId>
35     <artifactId>kernel</artifactId>
36     <version>7.2.5</version>
37   </dependency>
38   <dependency>
39     <groupId>com.itextpdf</groupId>
40     <artifactId>layout</artifactId>
41     <version>7.2.5</version>
42   </dependency>
43   <dependency>
44     <groupId>com.itextpdf</groupId>
45     <artifactId>io</artifactId>
46     <version>7.2.5</version>
47   </dependency>
48 </dependencies>
```

```
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.11.0</version>
      <configuration>
        <source>17</source>
        <target>17</target>
      </configuration>
    </plugin>
    <plugin>
      <groupId>org.openjfx</groupId>
      <artifactId>javafx-maven-plugin</artifactId>
      <version>0.0.8</version>
      <configuration>
        <mainClass>com.traductor.view.InterfazTraductor</mainClass>
      </configuration>
    </plugin>
  </plugins>
</build>
```

5 Flujo de ramas de GitHub Flow

- **feature/traductor**

Propósito: Desarrollo e implementación de la lógica del Modelo del traductor: TraductorBraille, DiccionarioBraille, SimboloBraille y ITraductor.

Regla: Debe contener las reglas de negocio para la transcripción de español a braille, no debe incluir código de la interfaz.

- **feature/interfaz**

Propósito: Desarrollo de la Vista y la lógica de los controladores de la interfaz JavaFX.

Regla: Implementar los componentes que permiten al usuario ingresar texto y ver la traducción en Braille, y también debe incluir las clases InterfazTraductor y TraductorController.

- **feature/traductorBrailleEspañol:**

Proposito: Desarrollar las funcionalidades destina a la generación de documentos en PDF en espejo, adicionalmente la traducción de textos de Braille a español.

Reglas: Implementar los componentes que permitan al usuario imprimir el texto en formato de espejo, también debe permitir al usuario traducir de Braille a español.

ICCD633 CONSTRUCCIÓN Y EVOLUCIÓN DE SOFTWARE

Versión 1.2.0

- **feature/Señalética:**
Propósito: Desarrollar las funcionalidades destinada a la generación de documentos PDF tipo señalética dentro de la capa de servicios.
Regla: Se desarrollan las implementaciones relacionadas con la generación, diseño y lógica interna de documentos PDF.
- **Main**
Propósito: Contiene el código estable de la última versión de producción.
Regla: El merge a Main solo se permite desde la rama develop, no se permite el desarrollo directo.
- **documentación**
Propósito: Manejar actualizaciones a la documentación del proyecto.
Regla: Usada para subir documentación formal sin afectar directamente el código fuente de la aplicación.

6 Procedimiento de instalación

Secuencia que se debe que seguir para clonar el repositorio y ejecutar la aplicación por primera vez.

1. **Instalar prerequisites**
Instalar JDK de java versión 25.0.1
Instalar Git
Instalar el IDE de Visual Studio Code
2. **Clonar repositorio**
Ejecutar el siguiente comando de git desde la terminal en donde se desea almacenar el repositorio.
`git clone --repositorio`
3. **Configurar el proyecto en el IDE**
Abrir el proyecto en el IDE, asegurando que el IDE reconozca el archivo de construcción Maven
4. **Descargar dependencias**
Ejecutar el comando de construcción para descargar todas las librerías necesarias: `mvn clean install`
5. **Ejecutar la aplicación**
Ejecutar usando el siguiente comando de construcción:
Primero: `mvn clean compile`
Segundo: `mvn javafx:run`

7 Javadoc

Se utilizo la herramienta de Javadoc para generar documentación legible directamente desde el código fuente, la cual se debe aplicar a todas las clases, interfaces, métodos públicos y atributos importantes.

7.1 Etiquetas claves a documentar:

@param: Se usa para describir un parámetro de un método o constructor. Debe incluir el nombre del parámetro y una descripción de su uso.

@return: Describe el valor de retorno de un método.

@throws: Indica las excepciones que puede lanzar un método y proporciona una descripción de cada una.

@see: Proporciona enlaces a otras clases, métodos o paquetes relacionados.

@since: Se utiliza en JavaDoc para indicar desde qué versión de una clase, método o campo está disponible en la API.